

GrigliaTron — Checklist Operativa per Sviluppatore AI

Scopo: dividere lo sviluppo di GrigliaTron in consegne piccole, controllabili e verificabili.

Metodo: ogni step deve produrre una modifica funzionante, testabile e non distruttiva.

Regola principale: non implementare tutto insieme. Ogni consegna deve essere completata, testata e approvata prima di passare alla successiva.

0. Preparazione e sicurezza progetto

Step 0.1 — Backup versione attuale funzionante

Obiettivo: preservare la versione usata con successo durante l'evento reale.

Da fare:

- ☐ Creare branch o copia progetto denominata `grigliatron-v1-stable`.
- ☐ Verificare che la versione attuale parta in locale.
- ☐ Verificare login Google.
- ☐ Verificare lettura/scrittura Firestore.
- ☐ Verificare deploy Netlify attuale.
- ☐ Salvare screenshot della versione attuale come riferimento.

Criterio di consegna:

- ☐ Esiste una versione stabile recuperabile in caso di problemi.
-

Step 0.2 — Controllo configurazioni sensibili

Obiettivo: evitare perdita di credenziali o dati personali.

Da fare:

- ☐ Verificare che `.env` sia in `.gitignore`.
- ☐ Verificare che non ci siano chiavi Firebase hardcoded nel codice.
- ☐ Verificare che l'email admin venga letta da variabile ambiente.
- ☐ Verificare che dati reali evento/partecipanti non siano hardcoded.
- ☐ Preparare `.env.example` aggiornato.

Criterio di consegna:

- ☐ Il progetto è sicuro per evoluzione futura e pubblicazione GitHub.
-

1. Refactor UX mobile-first

Step 1.1 — Analisi layout attuale

Obiettivo: identificare dove nasce il problema di scroll e confusione mobile.

Da fare:

- ☐ Analizzare struttura attuale dashboard.
- ☐ Elencare tutte le card/sezioni esistenti.
- ☐ Identificare componenti riutilizzabili.
- ☐ Identificare componenti da spostare in tab o pagine interne.
- ☐ Non modificare ancora la logica Firestore.

Criterio di consegna:

- ☐ Report sintetico con elenco sezioni e nuova destinazione prevista.
-

Step 1.2 — Creazione struttura di navigazione

Obiettivo: sostituire dashboard infinita con navigazione chiara.

Da fare:

- ☐ Creare struttura con tab principali:
- ☐ Home
- ☐ Liste
- ☐ Sondaggi
- ☐ Spese
- ☐ Info
- ☐ Implementare bottom navigation mobile.
- ☐ Implementare navigazione desktop coerente.
- ☐ Mantenere URL e stato evento funzionanti.
- ☐ Non implementare ancora nuove funzioni.

Criterio di consegna:

- ☐ Da mobile si vedono subito le tab principali.
 - ☐ Non serve più scrollare tutta la dashboard per raggiungere le sezioni.
-

Step 1.3 — Home compatta

Obiettivo: creare la nuova schermata principale.

Da fare:

- ☐ Mostrare titolo evento.
- ☐ Mostrare data evento.

- ☐ Mostrare luogo sintetico.
- ☐ Mostrare countdown.
- ☐ Mostrare riepilogo personale utente:
- ☐ elementi presi in carico;
- ☐ task assegnati;
- ☐ eventuale quota stimata;
- ☐ eventuali sondaggi aperti.
- ☐ Mostrare card compatte per le sezioni:
- ☐ Cibo & Bevande;
- ☐ Attrezzatura;
- ☐ Cose da fare;
- ☐ Sondaggi;
- ☐ Quote & Pagamenti;
- ☐ Scontrini;
- ☐ Dove & Quando;
- ☐ Meteo;
- ☐ Aggiornamenti;
- ☐ Recensioni.

Criterio di consegna:

- ☐ Nella prima schermata mobile l'utente capisce cosa aprire.
- ☐ Ogni card mostra almeno titolo, icona e mini stato.

Step 1.4 — Spostamento sezioni esistenti nelle tab

Obiettivo: riorganizzare senza rompere le funzioni esistenti.

Da fare:

- ☐ Spostare Cibo & Bevande nella tab Liste.
- ☐ Spostare Attrezzatura nella tab Liste.
- ☐ Spostare Cose da fare nella tab Liste.
- ☐ Spostare Costi nella tab Spese.
- ☐ Spostare Dove & Quando nella tab Info.
- ☐ Spostare Meteo nella tab Info.
- ☐ Spostare Aggiornamenti nella tab Info.
- ☐ Spostare Recensioni nella tab Info o post-evento.
- ☐ Mantenere tutti i dati Firestore esistenti.

Criterio di consegna:

- ☐ Le funzioni precedenti sono ancora utilizzabili.
- ☐ La navigazione è più chiara su mobile.

2. Permessi modifica/cancellazione utente

Step 2.1 — Normalizzazione campi autore

Obiettivo: assicurarsi che ogni contenuto abbia autore e timestamp.

Da fare:

- [] Verificare `foodItems`.
- [] Verificare `gearItems`.
- [] Verificare `tasks`.
- [] Verificare `updates`.
- [] Verificare `expenses`.
- [] Verificare `reviews`.
- [] Aggiungere dove mancanti:
- [] `createdBy`
- [] `createdByName`
- [] `createdAt`
- [] `updatedAt`

Criterio di consegna:

- [] Ogni documento creato dall'utente è riconducibile al suo autore.
-

Step 2.2 — UI modifica/cancellazione propri elementi

Obiettivo: permettere a ogni utente di correggere i propri errori.

Da fare:

- [] Mostrare pulsante Modifica solo ad autore o admin.
- [] Mostrare pulsante Elimina solo ad autore o admin.
- [] Implementare modifica item Cibo & Bevande.
- [] Implementare modifica item Attrezzatura.
- [] Implementare modifica task.
- [] Implementare modifica aggiornamenti.
- [] Implementare modifica spese base.
- [] Implementare modifica recensioni.

Criterio di consegna:

- [] Un utente può modificare/cancellare ciò che ha inserito lui.
 - [] Un utente non può modificare/cancellare contenuti di altri.
 - [] Admin può modificare/cancellare tutto.
-

Step 2.3 — Regole Firestore coerenti

Obiettivo: allineare sicurezza database alla UI.

Da fare:

- ☐ Aggiornare Firestore rules.
- ☐ Permettere update/delete se autore o admin.
- ☐ Bloccare update/delete di altri utenti se non admin.
- ☐ Testare con almeno due account Google.

Criterio di consegna:

- ☐ La sicurezza non dipende solo dalla UI.
-

3. Sondaggi

Step 3.1 — Modello dati sondaggi

Obiettivo: introdurre struttura Firestore per i sondaggi.

Da fare:

- ☐ Creare collection `events/{eventId}/polls`.
- ☐ Definire campi:
 - ☐ `question`
 - ☐ `options`
 - ☐ `multipleChoice`
 - ☐ `status`
 - ☐ `createdBy`
 - ☐ `createdByName`
 - ☐ `createdAt`
 - ☐ `updatedAt`
- ☐ Creare subcollection `votes`.
- ☐ Ogni utente deve avere un solo documento voto per sondaggio.

Criterio di consegna:

- ☐ Il modello dati è pronto e documentato.
-

Step 3.2 — UI creazione sondaggio

Obiettivo: permettere creazione sondaggi semplice.

Da fare:

- ☐ Creare form nuovo sondaggio.
- ☐ Campo domanda.
- ☐ Lista opzioni dinamica.
- ☐ Pulsante aggiungi opzione.
- ☐ Toggle voto singolo/multiplo.
- ☐ Salvataggio Firestore.

- ☐ Validazione minima: almeno domanda e due opzioni.

Criterio di consegna:

- ☐ Un utente può creare un sondaggio funzionante.
-

Step 3.3 — Voto e risultati

Obiettivo: permettere ai partecipanti di votare e vedere i risultati.

Da fare:

- ☐ Mostrare sondaggi aperti.
- ☐ Consentire voto singolo.
- ☐ Consentire voto multiplo se abilitato.
- ☐ Salvare voto per userId.
- ☐ Consentire modifica del proprio voto.
- ☐ Calcolare risultati.
- ☐ Mostrare barre o percentuali.

Criterio di consegna:

- ☐ Ogni partecipante può votare una sola volta per sondaggio, modificando il voto se necessario.
-

Step 3.4 — Chiusura sondaggio e trasformazione in item

Obiettivo: collegare i sondaggi all'organizzazione concreta.

Da fare:

- ☐ Autore o admin può chiudere il sondaggio.
- ☐ Admin può eliminare sondaggio.
- ☐ Admin può trasformare opzione vincente in:
 - ☐ Cibo & Bevande;
 - ☐ Attrezzatura;
 - ☐ Cose da fare.

Criterio di consegna:

- ☐ Una decisione presa con sondaggio può diventare elemento operativo.
-

4. Offerte spesa / Volantini

Step 4.1 — Modello dati offerte

Obiettivo: predisporre link a volantini e offerte.

Da fare:

- [] Creare collection `events/{eventId}/shoppingLinks`.
- [] Campi:
- [] `title`
- [] `url`
- [] `storeName`
- [] `description`
- [] `category`
- [] `expiresAt`
- [] `createdBy`
- [] `createdByName`
- [] `createdAt`
- [] `updatedAt`

Criterio di consegna:

- [] Struttura pronta senza impattare la Home.
-

Step 4.2 — UI offerte spesa

Obiettivo: aggiungere funzione marginale ma utile.

Da fare:

- [] Inserire sezione dentro tab Spese o Liste.
- [] Form aggiunta link.
- [] Apertura link in nuova scheda.
- [] Modifica/elimina da autore o admin.
- [] Badge opzionale "offerte disponibili" nella Home.

Criterio di consegna:

- [] Gli utenti possono condividere volantini senza appesantire la dashboard.
-

5. Costi & Scontrini con foto

Step 5.1 — Abilitazione Storage e struttura file

Obiettivo: preparare Firebase Storage per scontrini.

Da fare:

- [] Verificare Firebase Storage attivo.
- [] Definire path `events/{eventId}/receipts/{expenseId}/{filename}`.
- [] Aggiornare Storage rules.
- [] Aggiornare `.env.example` se necessario.

Criterio di consegna:

- [] Storage pronto e sicuro.
-

Step 5.2 — Upload scontrino

Obiettivo: permettere caricamento foto da mobile.

Da fare:

- [] Aggiungere input file nella form spesa.
- [] Consentire foto da camera/galleria smartphone.
- [] Upload su Firebase Storage.
- [] Salvare `receiptUrl` e `receiptPath` nella spesa.
- [] Gestire stato caricamento.
- [] Gestire errori upload.

Criterio di consegna:

- [] Un utente può registrare una spesa con foto scontrino.
-

Step 5.3 — Anteprima scontrino

Obiettivo: rendere controllabile la foto caricata.

Da fare:

- [] Mostrare miniatura nella card spesa.
- [] Aggiungere pulsante "Apri scontrino".
- [] Creare modal fullscreen.
- [] Mostrare immagine grande.
- [] Pulsante chiudi evidente.
- [] Layout mobile corretto.

Criterio di consegna:

- [] Chiunque abbia accesso all'evento può controllare visivamente lo scontrino.
-

Step 5.4 — Modifica/eliminazione spesa con scontrino

Obiettivo: gestire correzioni senza lasciare file inutili.

Da fare:

- [] Autore/admin può modificare importo, descrizione, note.
- [] Autore/admin può sostituire foto scontrino.
- [] Autore/admin può eliminare spesa.

- ☐ Se una spesa viene eliminata, eliminare anche file Storage collegato, se possibile.

Criterio di consegna:

- ☐ Gestione scontrini completa e pulita.
-

6. Quote & Pagamenti

Step 6.1 — Modello quote partecipanti

Obiettivo: introdurre logica quote flessibile.

Da fare:

- ☐ Aggiungere `quoteCount` ai partecipanti.
- ☐ Default `quoteCount = 1` per partecipante.
- ☐ Admin può modificare `quoteCount`.
- ☐ Supportare valori 0, 0.5, 1, 2, ecc.
- ☐ Aggiungere eventuale `manualTotalShares` nel documento evento.

Criterio di consegna:

- ☐ Il sistema può distinguere partecipanti reali e quote paganti.
-

Step 6.2 — Calcolo quote base

Obiettivo: calcolare quota singola e saldi.

Da fare:

- ☐ Calcolare totale spese.
- ☐ Calcolare totale quote.
- ☐ Se `manualTotalShares` esiste, usare quello.
- ☐ Calcolare quota singola.
- ☐ Calcolare totale anticipato per utente.
- ☐ Calcolare quota dovuta per utente.
- ☐ Calcolare saldo finale.

Criterio di consegna:

- ☐ La sezione mostra correttamente chi deve pagare e chi deve ricevere.
-

Step 6.3 — UI Quote & Pagamenti

Obiettivo: rendere comprensibili i conteggi.

Da fare:

- [] Card riepilogo;
- [] totale spese;
- [] totale quote;
- [] quota singola.
- [] Tabella/card partecipanti:
- [] nome;
- [] quoteCount;
- [] anticipato;
- [] dovuto;
- [] saldo;
- [] stato: deve pagare / deve ricevere / pari.
- [] Admin può modificare quoteCount.
- [] Admin può impostare manualTotalShares.

Criterio di consegna:

- [] Anche da mobile è chiaro quanto deve pagare ciascuno.
-

Step 6.4 — Algoritmo pagamenti consigliati

Obiettivo: suggerire chi deve pagare chi.

Da fare:

- [] Creare funzione `calculateSuggestedTransfers`.
- [] Input: utenti con saldo.
- [] Separare debitori e creditori.
- [] Generare trasferimenti progressivi.
- [] Arrotondare a 2 decimali.
- [] Mostrare lista trasferimenti.

Criterio di consegna:

- [] Il sistema propone trasferimenti pratici e coerenti.
-

Step 6.5 — PayPal link

Obiettivo: facilitare pagamento senza integrazione API complessa.

Da fare:

- [] Aggiungere campo `paypalLink` al partecipante.
- [] Utente o admin può impostarlo.
- [] Se destinatario ha PayPal link, mostrare pulsante "Paga con PayPal".
- [] Aprire link in nuova scheda.
- [] Non integrare API PayPal in questa fase.

Criterio di consegna:

- [] I pagamenti sono facilitati senza complessità tecnica.
-

7. Aggiornamenti e notifiche email

Step 7.1 — Tipologia aggiornamenti

Obiettivo: distinguere aggiornamenti normali, importanti e urgenti.

Da fare:

- [] Aggiungere campo `type` :
- [] info;
- [] important;
- [] urgent.
- [] Aggiornare UI form aggiornamenti.
- [] Mostrare badge visivo per importanti/urgent.
- [] Mostrare nella Home eventuali aggiornamenti urgenti.

Criterio di consegna:

- [] Gli aggiornamenti importanti emergono chiaramente.
-

Step 7.2 — Checkbox notifica email

Obiettivo: preparare invio email ai partecipanti.

Da fare:

- [] Aggiungere checkbox "Notifica i partecipanti via email".
- [] Default disattivata.
- [] Se tipo = urgent, attivarla automaticamente.
- [] Se notifiche non configurate, disabilitarla con tooltip.
- [] Aggiungere variabile `VITE_EMAIL_NOTIFICATIONS_ENABLED` .

Criterio di consegna:

- [] La UI è pronta per notifiche email controllate.
-

Step 7.3 — Preparazione collection mail

Obiettivo: integrare Firebase Trigger Email extension.

Da fare:

- [] Quando `notifyByEmail = true` , recuperare email partecipanti.

- [] Creare documento in collection `mail`.
- [] Struttura compatibile con Firebase Trigger Email.
- [] Impostare subject coerente.
- [] Impostare text e html.
- [] Aggiornare status notifica: `queued/sent/failed` se possibile.

Criterio di consegna:

- [] Il sistema prepara correttamente email da inviare tramite extension.
-

Step 7.4 — Documentazione notifiche

Obiettivo: rendere configurabile la funzione in open source.

Da fare:

- [] Aggiornare README.
- [] Spiegare installazione Firebase Trigger Email.
- [] Spiegare configurazione SMTP.
- [] Spiegare collection `mail`.
- [] Spiegare variabile ambiente.
- [] Spiegare limiti e sicurezza.

Criterio di consegna:

- [] Un altro sviluppatore può configurare le notifiche seguendo il README.
-

8. Foto evento e PDF ricordo

Step 8.1 — Stato evento concluso

Obiettivo: introdurre fase post-evento.

Da fare:

- [] Admin può impostare evento come concluso.
- [] Aggiungere campi:
- [] `closedAt`
- [] `closedBy`
- [] `status = closed`
- [] Home cambia modalità se evento concluso.
- [] Partecipanti vedono invito a lasciare recensione.

Criterio di consegna:

- [] L'app distingue chiaramente evento attivo ed evento concluso.
-

Step 8.2 — Notifica admin per foto evento

Obiettivo: chiedere all'admin di caricare la foto ricordo.

Da fare:

- [] Quando evento viene concluso, creare notifica/email per admin.
- [] Oggetto email: `[GrigliaTron] Evento concluso: carica la foto ricordo`.
- [] Inserire link evento se disponibile.
- [] Evitare invii duplicati.
- [] Campo `postEventPhotoRequested = true`.

Criterio di consegna:

- [] L'admin riceve richiesta chiara di caricare la foto evento.
-

Step 8.3 — Upload foto evento

Obiettivo: usare una foto ufficiale come immagine evento.

Da fare:

- [] Admin vede card "Carica foto ricordo evento".
- [] Upload immagine su Firebase Storage.
- [] Path `events/{eventId}/cover/event-cover.{ext}`.
- [] Salvare `coverImageUrl` e `coverImagePath` nel documento evento.
- [] Usare immagine come hero/cover dell'evento.
- [] Admin può sostituire immagine.

Criterio di consegna:

- [] La foto caricata diventa immagine principale dell'evento.
-

Step 8.4 — Pagina riepilogo PDF

Obiettivo: creare una vista HTML ordinata da esportare.

Da fare:

- [] Creare pagina/componente `EventSummaryPdfView`.
- [] Includere copertina con foto evento.
- [] Includere dati evento.
- [] Includere partecipanti.
- [] Includere cibo/bevande.
- [] Includere attrezzatura.
- [] Includere task.
- [] Includere spese e quote.
- [] Includere sondaggi e risultati.

- ☐ Includere aggiornamenti importanti.
- ☐ Includere recensioni.
- ☐ Stile pulito e stampabile.

Criterio di consegna:

- ☐ Esiste una vista riepilogativa bella e completa.
-

Step 8.5 — Generazione PDF

Obiettivo: generare PDF scaricabile.

Da fare:

- ☐ Valutare implementazione client-side con `html2canvas + jsPDF`.
- ☐ In alternativa predisporre `window.print()` con CSS print.
- ☐ Pulsante admin “Genera PDF evento”.
- ☐ Salvare eventualmente PDF su Storage.
- ☐ Aggiornare evento con `pdfUrl`, `pdfPath`, `pdfGeneratedAt`.
- ☐ Partecipanti possono scaricare PDF se disponibile e visibile.

Criterio di consegna:

- ☐ PDF generato e scaricabile come ricordo evento.
-

9. Open source readiness

Step 9.1 — Pulizia dati reali

Obiettivo: preparare progetto pubblico.

Da fare:

- ☐ Rimuovere eventi reali.
- ☐ Rimuovere nomi partecipanti reali.
- ☐ Rimuovere email personali dal codice.
- ☐ Rimuovere link Netlify personali.
- ☐ Rimuovere dati Firebase reali.
- ☐ Verificare `.env` escluso.

Criterio di consegna:

- ☐ Il repo non contiene dati personali o sensibili.
-

Step 9.2 — Documentazione base GitHub

Obiettivo: rendere il progetto comprensibile a terzi.

Da fare:

- ☐ README completo.
- ☐ Descrizione progetto.
- ☐ Screenshot.
- ☐ Funzioni principali.
- ☐ Installazione locale.
- ☐ Configurazione Firebase.
- ☐ Deploy Netlify.
- ☐ Roadmap.
- ☐ FAQ minima.

Criterio di consegna:

- ☐ Uno sviluppatore esterno può installare GrigliaTron seguendo il README.
-

Step 9.3 — File open source

Obiettivo: completare struttura pubblica.

Da fare:

- ☐ LICENSE MIT.
- ☐ CONTRIBUTING.md opzionale.
- ☐ docs/roadmap.md .
- ☐ docs/screenshots/ .
- ☐ .env.example aggiornato.
- ☐ firebase/firestore.rules .
- ☐ firebase/storage.rules .

Criterio di consegna:

- ☐ Il progetto è formalmente pronto per GitHub.
-

10. Sequenza consigliata di sviluppo

Consegna A — Stabilizzazione UX

- ☐ Step 0.1
- ☐ Step 0.2
- ☐ Step 1.1
- ☐ Step 1.2
- ☐ Step 1.3
- ☐ Step 1.4
- ☐ Step 2.1
- ☐ Step 2.2
- ☐ Step 2.3

Risultato: GrigliaTron v1.1, più usabile e mobile-first.

Consegna B — Decisioni collaborative

- ☐ Step 3.1
- ☐ Step 3.2
- ☐ Step 3.3
- ☐ Step 3.4
- ☐ Step 4.1
- ☐ Step 4.2

Risultato: GrigliaTron v1.2, con sondaggi e offerte spesa.

Consegna C — Spese reali e scontrini

- ☐ Step 5.1
- ☐ Step 5.2
- ☐ Step 5.3
- ☐ Step 5.4

Risultato: GrigliaTron v1.3, con scontrini fotografici.

Consegna D — Quote e pagamenti

- ☐ Step 6.1
- ☐ Step 6.2
- ☐ Step 6.3
- ☐ Step 6.4
- ☐ Step 6.5

Risultato: GrigliaTron v1.4, con quote, saldi e PayPal link.

Consegna E — Notifiche

- ☐ Step 7.1
- ☐ Step 7.2
- ☐ Step 7.3
- ☐ Step 7.4

Risultato: GrigliaTron v1.5, con aggiornamenti notificabili via email.

Consegna F — Ricordo evento

- ☐ Step 8.1
- ☐ Step 8.2
- ☐ Step 8.3
- ☐ Step 8.4
- ☐ Step 8.5

Risultato: GrigliaTron v1.6, con foto evento e PDF ricordo.

Consegna G — Open source release

- ☐ Step 9.1
- ☐ Step 9.2
- ☐ Step 9.3

Risultato: GrigliaTron v2.0, pronto per GitHub.

11. Regole per lo sviluppatore AI

Ogni consegna deve rispettare queste regole:

- ☐ Non rompere login Google.
 - ☐ Non rompere Firestore.
 - ☐ Non rompere deploy Netlify.
 - ☐ Non hardcodare email admin.
 - ☐ Non inserire credenziali nel codice.
 - ☐ Non cancellare dati esistenti senza migrazione.
 - ☐ Ogni nuova funzione deve essere testabile singolarmente.
 - ☐ Ogni modifica deve indicare file modificati.
 - ☐ Ogni consegna deve chiudersi con `npm run build` riuscito.
 - ☐ Ogni consegna deve includere mini report finale.
-

12. Prompt breve per iniziare la prossima consegna

Implementa solo la Consegna A di GrigliaTron.

Obiettivo:

Rendere GrigliaTron mobile-first, eliminando la dashboard infinita e introducendo Home compatta, bottom navigation e permessi di modifica/cancellazione per i contenuti creati dall'utente.

Non implementare ancora sondaggi, scontrini, quote avanzate, notifiche email, PDF o open source cleanup.

Step inclusi:

- 0.1 Backup versione stabile
- 0.2 Controllo configurazioni sensibili
- 1.1 Analisi layout attuale
- 1.2 Creazione struttura navigazione
- 1.3 Home compatta
- 1.4 Spostamento sezioni esistenti nelle tab
- 2.1 Normalizzazione campi autore
- 2.2 UI modifica/cancellazione propri elementi

- 2.3 Regole Firestore coerenti

Vincoli:

- Non rompere Firebase Auth.
- Non rompere Firestore.
- Non rompere deploy Netlify.
- Non modificare inutilmente lo schema dati esistente.
- Ogni utente può modificare/cancellare solo ciò che ha creato.
- Admin può modificare/cancellare tutto.

Output richiesto:

- elenco file modificati;
- spiegazione della nuova navigazione;
- conferma build npm run build;
- eventuali note su migrazione dati;
- indicazioni per test manuale mobile e desktop.